SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week6 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number: 11.1(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 11 – Data Structures with AI: Implementing Fundamental Structures** **Lab Objectives** • Use AI to assist in designing and implementing fundamental data	Week6 - Monday

structures in Python.

- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

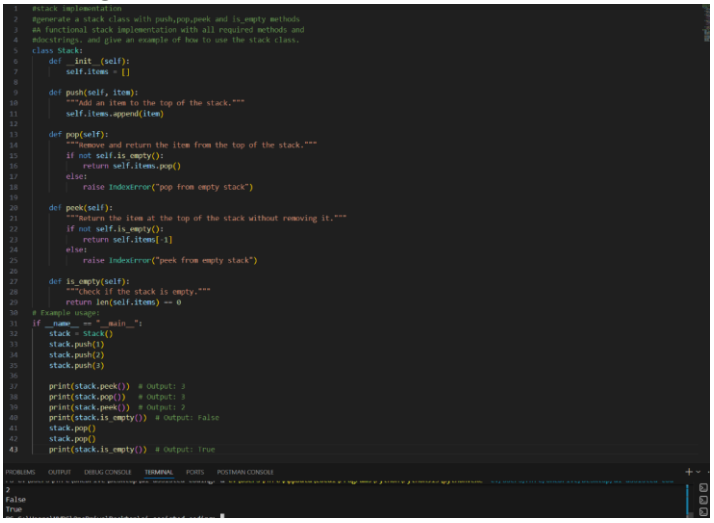
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.



```
#stack implementation
#generate a stack class with push,pop,peek and is_empty methods
#A functional stack implementation with all required methods and
#docstrings. and give an example of how to use the stack class.
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the item from the top of the stack."""
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("pop from empty stack")

    def peek(self):
        """Return the item at the top of the stack without removing it."""
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("peek from empty stack")

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print(stack.peek())  # Output: 3
    print(stack.pop())  # Output: 3
    print(stack.peek())  # Output: 2
    print(stack.is_empty())  # Output: False
    stack.pop()
    stack.pop()
    print(stack.is_empty())  # Output: True
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE
2
False
True
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```python
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item from the front of the queue."""
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("dequeue from empty queue")

    def peek(self):
        """Return the item at the front of the queue without removing it."""
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("peek from empty queue")

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.peek())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.peek())  # Output: 2
    print(queue.size())  # Output: 2
    queue.dequeue()
    queue.dequeue()
    print(queue.is_empty())  # Output: True
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
2
True
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
    pass

class LinkedList:
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

```python
#linked list
#generate a Singly Linked List with insert and display methods.
##a working linked list implementation with clear method documentation. give an example of how to use the linked list class.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the linked list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Example usage:
if __name__ == "__main__":
    linked_list = Linkedlist()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)

    print("Linked List contents:")
    linked_list.display()  # Output: 1 2 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/MYPC/OneDri
ing/ai_ass.11.1/task.03.py"
1 2 3
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

```python
#binary search
#create a BST with insert and in-order traversal methods.
#BST implementation with recursive insert and traversal methods. and give an example of how to use the BST class.
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        """Insert a new node with the given key into the BST."""
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursive(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert_recursive(node.right, key)

    def in_order_traversal(self):
        """Perform in-order traversal of the BST and return a list of values."""
        return self._in_order_recursive(self.root)

    def _in_order_recursive(self, node):
        res = []
        if node:
            res = self._in_order_recursive(node.left)
            res.append(node.val)
            res = res + self._in_order_recursive(node.right)
        return res
# Example usage:
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)

    print("In-order Traversal of BST:", bst.in_order_traversal())  # Output: [2, 3, 4, 5, 7]
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

1 2 3
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

ai_ass.11.1 > task.05.py > ...

```python
class HashTable:
    def search(self, key):
            if k == key:
                return v  # Return the value if the key is found
        return None  # Return None if the key is not found

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True  # Return True if deletion was successful
        return False  # Return False if the key was not found
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    print(hash_text.search("city"))  # Output: New York

    hash_table.delete("age")
    print(hash_table.search("age"))   # Output: None
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\
New York
None
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

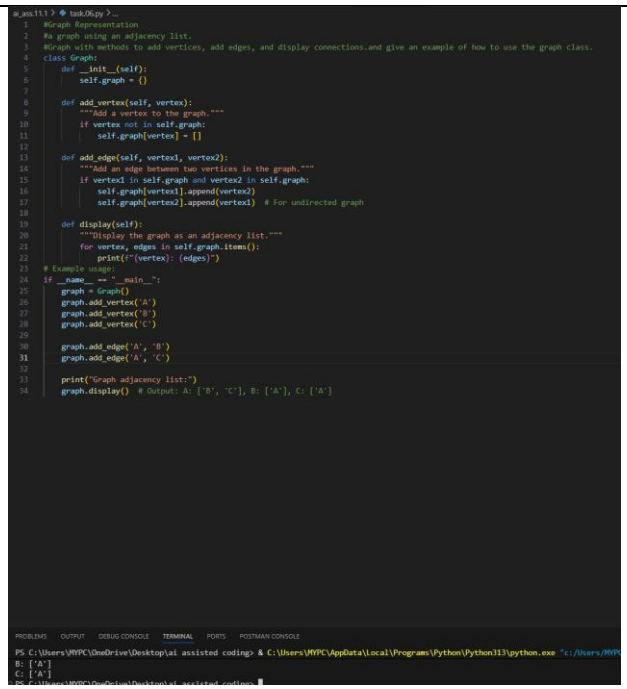
Sample Input Code:

class Graph:

    pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.



Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```python
#Priority Queue
#a graph using an adjacency list.
#Graph with methods to add vertices, add edges, and display connections.and give an example of how to use the graph class.
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        """Check if the priority queue is empty."""
        return not self.elements

    def put(self, item, priority):
        """Add an item to the priority queue with a given priority."""
        heapq.heappush(self.elements, (priority, item))
    def get(self):
        """Remove and return the item with the highest priority (lowest value)."""
        if not self.is_empty():
            return heapq.heappop(self.elements)[1]
        else:
            raise IndexError("get from empty priority queue")

    def peek(self):
        """Return the item with the highest priority without removing it."""
        if not self.is_empty():
            return self.elements[0][1]
        else:
            raise IndexError("peek from empty priority queue")

    def size(self):
        """Return the number of items in the priority queue."""
        return len(self.elements)
# Example usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.put("task1", priority=3)
    pq.put("task2", priority=1)
    pq.put("task3", priority=2)

    print(pq.peek())  # Output: task2 (highest priority)
    print(pq.get())   # Output: task2
    print(pq.peek())  # Output: task3 (next highest priority)
    print(pq.size())  # Output: 2
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\Programs\Python\Python313\python.exe "c:/Users
task3
2
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

```
class DequeDS:
    pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

ai_ass.11.1 > task.08.py > ...

```python
class Deque:
    def peek_front(self):
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("peek from empty deque")

    def peek_rear(self):
        """Return the item at the rear of the deque without removing it."""
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("peek from empty deque")

    def size(self):
        """Return the number of items in the deque."""
        return len(self.items)

    def is_empty(self):
        """Check if the deque is empty."""
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    deque = Deque()
    deque.add_rear(1)
    deque.add_rear(2)
    deque.add_front(0)

    print(deque.peek_front())  # Output: 0
    print(deque.peek_rear())   # Output: 2
    print(deque.remove_front())  # Output: 0
    print(deque.remove_rear())   # Output: 2
    print(deque.size()) # Output: 1
    print(deque.is_empty()) # Output: False
    deque.remove_front()
    print(deque.is_empty()) # Output: True
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\Programs\P
False
True
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their

	due dates.
	4. Bus Scheduling System – Maintain bus routes and stop connections.
	5. Cafeteria Order Queue – Serve students in the order they arrive.
	Student Task:
	• For each feature, select the most appropriate data structure from the list below:
	  ○ Stack
	  ○ Queue
	  ○ Priority Queue
	  ○ Linked List
	  ○ Binary Search Tree (BST)
	  ○ Graph
	  ○ Hash Table
	  ○ Deque
	• Justify your choice in 2–3 sentences per feature.
	• Implement one selected feature as a working Python program with AI-assisted code generation.
	Expected Output:
	• A table mapping feature → chosen data structure → justification.
	• A functional Python program implementing the chosen feature with comments and docstrings.	

ai_ass.11.1 > task.09.py > ...

```python
#Real-Time Application Challenge – Choose the Right Data Structure
"""
Cafeteria Queue Management System
Implements FIFO ordering using Queue concept.
"""

from collections import deque

class CafeteriaQueue:
    def __init__(self):
        # Initialize empty queue
        self.orders = deque()

    def add_order(self, name):
        """Add a student's order to the queue."""
        self.orders.append(name)
        print(f"{name} joined the queue.")

    def serve_order(self):
        """Serve the next student."""
        if self.orders:
            student = self.orders.popleft()
            print(f"Serving {student}")
        else:
            print("No students waiting.")

    def show_queue(self):
        """Display current queue."""
        print("Current Queue:", list(self.orders))


# Example execution
queue = CafeteriaQueue()

queue.add_order("Arjun")
queue.add_order("Meena")
queue.add_order("Kiran")

queue.show_queue()

queue.serve_order()
queue.show_queue()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\Programs\Python\Python3
Serving Arjun
Current Queue: ['Meena', 'Kiran']
```

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
Shopping Cart Management System
Using Linked List Data Structure
"""

class Node:
    """
    Node class represents each product in the cart
    """
    def __init__(self, product_name):
        self.product_name = product_name
        self.next = None


class ShoppingCart:
    """
    Linked list implementation for Shopping Cart
    Allows adding, removing, and displaying products
    """

    def __init__(self):
        self.head = None

    def add_product(self, product_name):
        """
        Add a product to the shopping cart
        """
        new_node = Node(product_name)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node

        print(product_name, "added to cart.")

    def remove_product(self, product_name):
        """
        Remove a product from the cart
        """
        temp = self.head
        prev = None

        while temp:
            if temp.product_name == product_name:
                if prev is None:
                    self.head = temp.next
                else:
                    prev.next = temp.next

                print(product_name, "removed from cart.")
                return
```

```python
class ShoppingCart:
    def remove_product(self, product_name):

            prev = temp
            temp = temp.next

        print(product_name, "not found in cart.")

    def display_cart(self):
        """
        Display all items in the cart
        """
        if self.head is None:
            print("Cart is empty")
            return

        temp = self.head
        print("\nProducts in Cart:")
        while temp:
            print("-", temp.product_name)
            temp = temp.next


# Example usage
cart = ShoppingCart()

cart.add_product("Laptop")
cart.add_product("Headphones")
cart.add_product("Mouse")

cart.display_cart()

cart.remove_product("Headphones")

cart.display_cart()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C
- Laptop
- Mouse
```